

电子科技大学

课程实验

一、实验项目名称：构造语法分析器

二、实验目的

1. 理解递归下降语法分析的思想与实现步骤。
2. 学会设计数据结构存储变量表、过程表，并掌握语法错误处理方法。

三、实验内容

1. 使用 C 语言设计、编制并调试一个语法分析程序，并用它对词法分析程序所提供的单词序列进行语法检查和结构分析。
2. 输入：词法分析生成的二元式文件。
3. 输出：
 - 变量名表文件 (*.var)，包含变量名、所属过程、分类、变量类型、变量层次、变量再变量表中的位置；
 - 过程名表文件 (*.pro)，包含过程名、过程类型、层次及变量区间信息；
 - 错误文件 (*.err)，记录语法错误信息。
4. 要求：
 - 使用递归下降法实现，保证文法无左递归、无公共左因子；
 - 对语法结构进行正确的匹配与错误处理。

四、实验结果：

1. 压缩包：源代码、运行结果
2. 实验报告

五、测试程序（MINI 语言）

测试程序一：

```
begin
  integer k;
  integer function F(n);
  begin
```

```

integer n;
if n<=0 then F:=1
else F:=n*F(n-1)
end;
read(m);
k:=F(m);
write(k)
end

```

测试程序二：

```

begin
integer x;
integer y
read(x);
if x > 0 then
y := x * 2;
else
end
edn

```

六、输出文件格式

1. *.err —— 错误信息

格式：

LINE:行号[空格]错误性质； 如果无错误, 则*.err 为空

例如：

LINE:7 Missing ';' before 'else'

2. *.var —— 变量名表（每行“vname vproc vkind vtype vlev vadr”）

变量名 vname: char(16)

所属过程 vproc:char(16)

分类 vkind: 0..1(0—变量、1—形参)

变量类型 vtype: types

变量层次 vlev: int

变量在变量表中的位置 vadr: int(相对第一个变量而言)

3. *.pro —— 过程名表（每行“pname ptype plev fadr ladr”）

过程名 pname: char(16)

过程类型 ptype: types

过程层次 plev: int

第一个变量在变量表中的位置 fadr: int

最后一个变量在变量表中的位置 ladr: int

七、实验过程和说明

1. Mini 语言文法

```
<程序>→<分程序>
<分程序>→begin <说明语句表>; <执行语句表> end
<说明语句表>→<说明语句> | <说明语句表> ; <说明语句>
<说明语句>→<变量说明> | <函数说明>
<变量说明>→integer <变量>
<变量>→<标识符>
<标识符>→<字母> | <标识符><字母> | <标识符><数字>
<字母>→a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u
| v | w | x | y | z
<数字>→0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<函数说明>→integer function <标识符> (<参数>); <函数体>
<参数>→<变量>
<函数体>→begin <说明语句表>; <执行语句表> end
<执行语句表>→<执行语句> | <执行语句表>; <执行语句>
<执行语句>→<读语句> | <写语句> | <赋值语句> | <条件语句>
<读语句>→read(<变量>)
<写语句>→write(<变量>)
<赋值语句>→<变量>:=<算术表达式>
<算术表达式>→<算术表达式>-<项> | <项>
<项>→<项>*<因子> | <因子>
<因子>→<变量> | <常数> | <函数调用>
<常数>→<无符号整数>
<无符号整数>→<数字> | <无符号整数><数字>
<条件语句>→if<条件表达式>then<执行语句>else <执行语句>
<条件表达式>→<算术表达式><关系运算符><算术表达式>
<关系运算符> →< | <= | > | >= | = | <>
```

2. 语法分析递归下降分析器的构造

当改造文法为无公共左因子,无左递归时,让每个非终结符对应子程序;该过程对相应的非终结符产生式的右部短语进行语法分析,例如:

```
G(E)      E→E+T | T
          T→T*F | F
          F→(E) | i
```

Step1: 改造文法, 消除左递归:

```
E→TE'
E'→+TE' | ε
T→FT'
T'→*FT' | ε
F→(E) | i
```

Step2: 写匹配函数, 检查当前输入符号是不是期待的 token, 若是就读下一个,

```

procedure match(t:token);
begin
    if lookahead=t then lookahead=nexttoken
    else error
end;
本例中：
procedure E;
begin
    T;
    E'
end;

procedure T;
begin
    F;
    T'
end;

procedure E';
    if lookahead='+' then begin match('+');
        T;
        E'
    end;

procedure T';
    if lookahead='*' then begin match('*');
        F;
        T'
    end;

procedure F;
    if lookahead='i' then match('i')
    else if lookahead='(' then
        begin
            match('(');
            E;
            if lookahead=')' then match(')')
            else error
        end
    else error;
i+i*i#的递归下降分析过程：

```

